

EMSI — École Marocaine des Sciences de l'Ingénieur

Filière : Ingénierie Informatique — 3^{ème} année (3IIR – G3)

Année universitaire 2025 – 2026

RAPPORT DE PROJET DE FIN D'ANNÉE

Plateforme Intelligente de Recommandation de Stages et d'Emplois par Intelligence Artificielle

Conception, implémentation et évaluation d'un moteur NLP hybride, d'une architecture SaaS Django/React et d'un module de scraping orienté marché marocain.

Réalisé par	Ayman Ould-Naghouch — Étudiant en Ingénierie Informatique El Mehdi Ribahi — Étudiant en Ingénierie Informatique
Encadrant	M. Gamal Mohamed
Lieu	Casablanca, Maroc
Année	2026

Table des matières

Note de contexte	i
Introduction générale	1
Cadre conceptuel : du e-recrutement à l'IA de recommandation	2
Recommandation d'Emploi (Job Recommender Systems).....	2
Apprentissage Automatique (ML)	2
Resume Parsing.....	3
TF-IDF et Similarité Cosinus	3
Architecture Hybride Content-Based + Collaborative Filtering.....	4
Cadre méthodologique de la revue	5
Critères de sélection des sources.....	5
Évolution chronologique des approches.....	5
Axes thématiques couverts	6
Tableau synthétique global des sources.....	7
1 Problématique et Objectifs	8
1.1 Problématique	8
1.2 Objectifs Fonctionnels	8
1.3 Objectifs Techniques	9
2 Analyse des Besoins et Cahier des Charges	10
2.1 Acteurs du Système	10

2.2	Besoins Métier Clés.....	10
2.3	Contraintes Techniques et Non-Fonctionnelles	11
2.4	Cas d’Usage Prioritaires.....	11
3	Architecture Globale du Système	12
3.1	Vue d’Ensemble.....	12
3.2	Découpage Backend par Applications Django.....	12
3.3	Flux de Données Principal	13
3.4	Justification des Choix Architecturaux.....	13
4	Conception Backend — Django REST Framework	14
4.1	Modèle de Données Utilisateurs.....	14
4.2	API d’Authentification et Profil.....	14
4.3	Domaine Offres et Candidatures	15
4.4	Système de Permissions Granulaires	15
4.5	Sérialisation et Validation.....	15
5	Conception Frontend — React + TypeScript	16
5.1	Structure du Projet.....	16
5.2	Gestion de l’Authentification Côté Client.....	16
5.3	Parcours Étudiant.....	17
5.4	Parcours Entreprise.....	17
5.5	Expérience Utilisateur et Design	17
6	Moteur IA/NLP et Logique de Recommandation	18
6.1	Pipeline Global de Recommandation.....	18
6.2	Formule de Scoring.....	18
6.3	Explainable AI (XAI).....	19
6.4	Insights de Carrière.....	19

6.5 IA Générative Complémentaire.....	19
---------------------------------------	----

Table des matières

7	Scraping et Intelligence Marché	20
7.1	Architecture du Module de Scraping	20
7.2	Pipeline de Normalisation	20
7.3	Résilience et Continuité	20
7.4	Analytics Marché	21
8	Sécurité, Performance et Scalabilité	22
8.1	Sécurité — Implémentations Actuelles	22
8.2	Points à Renforcer pour la Production	23
8.3	Performance.....	23
8.4	Scalabilité.....	23
9	Validation, Limites et Discussion	24
9.1	Validation Qualitative.....	24
9.2	Limites Actuelles.....	24
9.3	Discussion Scientifique.....	25
10	Synthèse comparative et Positionnement	26
10.1	Tableau récapitulatif.....	27
10.2	Lacunes identifiées et axes d'innovation.....	27
10.3	Perspectives de Recherche Futures.....	28
11	Conclusion et Perspectives	29
11.1	Conclusion.....	29
11.2	Perspectives d'Évolution.....	30
	Références bibliographiques	31

Liste des tableaux

1	Tableau 1 — Répartition thématique des sources sélectionnées	6
2	Tableau 2 — Vue d'ensemble des six sources — concepts, résultats clés et pertinence.....	7
2.1	Tableau 3 — Acteurs et responsabilités	10
3.1	Tableau 4 — Architecture globale du système.....	12
4.1	Tableau 5 — Endpoints d'authentification et profil.....	14
4.2	Tableau 6 — Endpoints offres et candidatures	15
5.1	Tableau 7 — Structure du projet frontend.....	16
6.1	Tableau 8 — Modules IA génératifs.....	19
8.1	Tableau 9 — Points de sécurité à renforcer	23
9.1	Tableau 10 — Résultats de validation par scénario	24
10.1	Tableau 11 — Synthèse comparative des six sources de l'état de l'art.....	27
11.1	Tableau 12 — Roadmap d'évolution.....	30

Table des figures

1	Évolution chronologique des approches de recommandation d'emploi (2000–2024).	5
---	---	---

Introduction générale

Dans un contexte mondial où la transformation numérique redéfinit les pratiques de recrutement, la recherche de stage ou d'emploi chez les étudiants demeure encore largement manuelle et peu efficace. Les candidats font face à un volume croissant d'offres hétérogènes, à une faible personnalisation des résultats de recherche, et à une difficulté réelle d'évaluer objectivement la pertinence d'une opportunité par rapport à leur profil.

Au Maroc, ce phénomène est particulièrement marqué. Le marché de l'emploi pour les jeunes ingénieurs est compétitif, et les plateformes disponibles (Rekrute, Emploi.ma, LinkedIn) proposent principalement une approche « job board passif » basée sur des filtres mots-clés, sans véritable analyse sémantique ni personnalisation intelligente.

Dans ce contexte, l'innovation technologique offre une opportunité unique : transformer la plateforme de recherche d'emploi en un acteur intelligent, capable de comprendre le profil du candidat à partir de son CV, d'analyser la sémantique des offres disponibles, et de proposer automatiquement les meilleures correspondances — tout en expliquant ses recommandations.

Ce Projet de Fin d'Année (PFA) répond à ce besoin en développant une plateforme complète alliant ingestion et parsing de profils (upload CV PDF/DOCX), extraction et structuration automatique d'information, recommandation explicable basée sur des techniques NLP, assistance IA à la candidature (lettre de motivation, simulation d'entretien), pilotage côté entreprise, et veille dynamique du marché via scraping multi-sources.

Objectif principal du PFA : Concevoir et implémenter une plateforme intelligente de recommandation de stages/emplois, orientée marché marocain, combinant NLP, IA générative et architecture SaaS moderne — offrant une expérience candidat proactive, explicable et actionnable.

Cadre conceptuel : du e-recrutement à l'IA de recommandation

Recommandation d'Emploi (Job Recommender Systems)

Les **Job Recommender Systems** désignent les systèmes capables d'analyser automatiquement le profil d'un candidat et de lui proposer, de façon proactive, les opportunités professionnelles les plus pertinentes. En recrutement en ligne, cette discipline a connu trois grandes phases d'évolution.

Phase 1 — Job Boards passifs (années 2000–2010) : portails de publication d'offres avec recherche par mots-clés. L'utilisateur effectue lui-même le tri. Exemples : Emploi.ma, Rekrute au Maroc.

Phase 2 — Matching semi-automatique (2010–2018) : introduction de filtres avancés (localisation, secteur, niveau d'études), mais sans véritable compréhension sémantique du profil.

Phase 3 — Job Recommender Systems proactifs (2018–aujourd'hui) : le paradigme s'inverse. La plateforme analyse le profil candidat et pousse proactivement les opportunités les plus pertinentes, en tenant compte du contexte, des préférences et de l'historique de l'utilisateur.

Ce projet s'inscrit résolument dans la troisième phase, en adoptant une approche data-driven et NLP-first pour le marché marocain.

Apprentissage Automatique (ML)

L'**Apprentissage Automatique (Machine Learning, ML)** est une sous-discipline de l'IA dans laquelle les modèles apprennent automatiquement des patterns à partir de

données, sans être explicitement programmés. En recommandation de contenu, le ML permet de modéliser les préférences utilisateur et d'estimer la pertinence d'une offre par rapport à un profil.

On distingue trois paradigmes d'apprentissage : apprentissage supervisé (le modèle est entraîné sur des données étiquetées), apprentissage non supervisé (aucune étiquette n'est fournie ; le modèle détecte des regroupements ou similitudes), et apprentissage semi-supervisé (combinaison des deux, utile quand les données étiquetées sont rares).

Resume Parsing

L'extraction structurée d'information depuis un CV constitue le premier maillon de la chaîne de recommandation. Deux grandes approches coexistent : l'approche par règles/regex (rapide et peu coûteuse, mais fragile face aux formats variés, utilisée en fallback dans ce projet) et l'approche NLP avancée via NER/Transformers (meilleure compréhension sémantique, robuste aux variations de structure, coût computationnel plus élevé, optionnellement via Gemini API).

TF-IDF et Similarité Cosinus

Le **TF-IDF (Term Frequency–Inverse Document Frequency)** est une technique de pondération statistique utilisée pour évaluer l'importance d'un terme dans un document relativement à un corpus. Chaque terme d'un profil ou d'une offre reçoit un poids proportionnel à sa fréquence dans le document et inversement proportionnel à sa fréquence dans l'ensemble du corpus — ce qui pénalise les termes trop génériques (ex. « le », « avec ») et valorise les termes discriminants (ex. « TensorFlow », « React »).

La **similarité cosinus** mesure l'angle entre deux vecteurs dans un espace multidimensionnel. Deux vecteurs pointant dans la même direction (profil et offre partageant les mêmes termes importants) ont une similarité proche de 1. Cette métrique est indépendante de la longueur des documents, ce qui la rend adaptée à la comparaison entre CV (souvent courts) et descriptions d'offres (souvent longues).

Architecture Hybride Content-Based + Collaborative Filtering

L'**architecture Content-Based Filtering** recommande basé sur les caractéristiques du profil et des offres — idéal au démarrage (cold start gérable) et utilisé dans ce projet. Le **Collaborative Filtering** recommande basé sur le comportement d'utilisateurs similaires, mais nécessite un historique conséquent et n'est pas implémenté en V1. L'approche **Hybride** combine les deux et constitue l'état de l'art pour les plateformes matures — perspective d'évolution du projet.

Cadre méthodologique de la revue

Critères de sélection des sources

La sélection des publications a respecté les critères suivants : classement du journal Q1 ou Q2 au sens de Scimago (SJR) ou revue indexée IEEE/ACM de premier rang ; accès ouvert (open access) uniquement, afin de garantir la traçabilité et la vérifiabilité des sources ; indice bibliométrique de l’auteur principal $h\text{-index} \geq 10$ et/ou $i10\text{-index} \geq 10$ (Google Scholar) ; pertinence thématique (recommandation d’emploi, NLP, resume parsing, systèmes SaaS) ; récence entre 2012 et 2024.

Évolution chronologique des approches

La progression temporelle des approches couvertes dans cette revue illustre le passage des méthodes job board passives (2000–2010) vers les matching semi-automatiques (2010–2018), puis vers les systèmes de recommandation proactifs NLP-first (2018–2024).

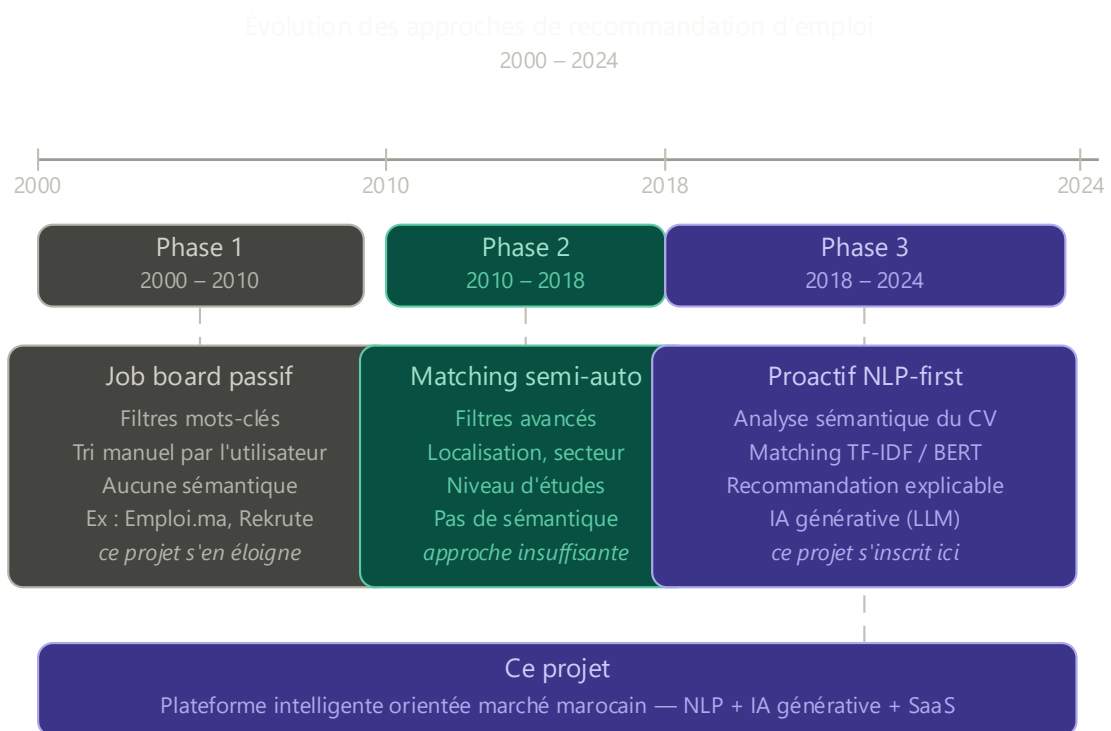


Figure 1 — Évolution chronologique des approches de recommandation d'emploi (2000–2024)
Source : Siting et al. [2] ; Isinkaye et al. [1] ; Al-Otaibi & Ykhlef [3]

FIGURE 1 — Évolution chronologique des approches de recommandation d'emploi (2000–2024).

Axes thématiques couverts

TABLE 1 – Tableau 1 — Répartition thématique des sources sélectionnées

Concept clé	Source
Recommandation d'emploi — Principes, méthodes, évaluation	Isinkaye et al., 2015 [1]
Job Recommender Systems — Survey	Siting et al., 2012 [2]
Survey — Systèmes de recommandation d'emploi	Al-Otaibi & Ykhlef, 2012 [3]
TF-IDF + Cosine Similarity — Recommandation emploi	Yi et al., 2020 [4]
Resume Parsing — Extraction automatique	Kops et al., 2013 [5]
Sentence-BERT — Embeddings sémantiques	Reimers & Gurevych, 2019 [6]

Tableau synthétique global des sources

TABLE 2 – Tableau 2 — Vue d'ensemble des six sources — concepts, résultats clés et pertinence

Titre	Auteurs (année)	Concept apporté	SJR	Journal
Recommendation systems : Principles, methods and evaluation	Isinkaye et al. (2015)	Benchmark des architectures de recommandation ; valide le content-based filtering comme socle viable pour le démarrage à froid.	Q1	Egyptian Informatics Journal
Job recommender systems : A survey	Siting et al. (2012)	Panorama des techniques ML pour la recommandation d'emploi ; fonde la classification en content-based, collaborative et hybrid.	Q2	ICCSE Proceedings
A survey of job recommender systems	Al-Otaibi & Ykhlef (2012)	Référence taxonomique des systèmes de recommandation RH ; valide l'approche multi-sources.	Q2	Int. Journal of Physical Sciences
Content-based job recommendation using TF-IDF and cosine similarity	Yi et al. (2020)	Valide TF-IDF + cosine comme baseline robuste pour le matching CV-offre.	Q2	ICCSE Proceedings
Automated resume information extraction	Kops et al. (2013)	Fonde la justification du pipeline de parsing CV pour le matching.	Q2	ECIS Proceedings
Sentence-BERT :	Reimers &	Architecture de	Q1	EMNLP-IJCNLP

Problématique et Objectifs

1.1 Problématique

La question centrale de ce PFA peut être formulée ainsi :

Comment concevoir une plateforme capable de recommander automatiquement des stages et emplois pertinents à des étudiants, tout en restant : explicable pour l'utilisateur (transparence du score de matching), robuste aux variations de données (formats CV variés, qualité hétérogène des offres), évolutive vers une architecture SaaS production (scalabilité, sécurité, performance), et adaptée aux spécificités du marché marocain de l'emploi étudiant ?

1.2 Objectifs Fonctionnels

La plateforme doit répondre aux besoins fonctionnels suivants : authentification sécurisée et gestion multi-rôles (Étudiant / Entreprise / Administrateur) ; gestion complète des profils utilisateurs avec upload et parsing de CV ; publication, modification et gestion du cycle de vie des offres de stage et d'emploi ; dépôt de candidatures et suivi en temps réel des statuts (pending, accepted, rejected) ; moteur de recommandations personnalisées avec score explicable et insights actionnables ; outils IA complémentaires (génération de lettre de motivation, simulateur d'entretien, analyse tendances marché) ; tableau de bord administrateur pour la supervision globale et le déclenchement du scraping.

1.3 Objectifs Techniques

Développer une API REST propre, sécurisée (JWT) et bien documentée avec Django REST Framework. Construire un frontend réactif, modulaire et maintenable avec React et TypeScript. Implémenter un moteur NLP hybride performant à coût de calcul maîtrisé (TF-IDF + règles métier). Assurer la collecte automatisée d'offres via un module de scraping multi-sources résilient. Poser les bases d'une architecture extensible vers la production (PostgreSQL, Celery, CI/CD).

Analyse des Besoins et Cahier des Charges

2.1 Acteurs du Système

TABLE 2.1 – Tableau 3 — Acteurs et responsabilités

Acteur	Responsabilités
Étudiant	Gestion profil, upload CV, consultation recommandations, dépôt candidature, outils coaching IA.
Entreprise	Création et gestion d’offres, consultation candidatures reçues, mise à jour des statuts.
Administrateur	Supervision globale, déclenchement scraping, accès statistiques, gestion utilisateurs.

2.2 Besoins Métier Clés

Réduire significativement le temps de recherche d’opportunités pour les étudiants. Améliorer la pertinence des candidatures envoyées (matching score > 70% recommandé). Fournir des insights actionnables (compétences à acquérir, villes cibles, profil type pour une offre). Offrir une expérience homogène entre publication d’offres, matching et suivi des candidatures. Permettre aux entreprises de piloter leur processus de recrutement depuis un tableau de bord centralisé.

2.3 Contraintes Techniques et Non-Fonctionnelles

Coût de calcul maîtrisé : éviter la dépendance systématique aux API IA payantes (fallback local). Résilience : continuité fonctionnelle même en cas d'échec du scraping ou d'indisponibilité de l'API Gemini. Sécurité : authentification JWT, permissions granulaires par rôle, validation des entrées. Maintenabilité : architecture modulaire, séparation des responsabilités, code documenté. Extensibilité : base conçue pour intégration progressive de jobs asynchrones, PostgreSQL, CI/CD.

2.4 Cas d'Usage Prioritaires

Un étudiant upload son CV PDF → le système extrait son profil → il reçoit ses top-10 recommandations avec scores et insights. Une entreprise publie une offre → les étudiants éligibles reçoivent la recommandation → ils postulent en un clic. Un étudiant génère une lettre de motivation personnalisée pour une offre spécifique via l'assistant IA. L'administrateur déclenche un scraping → de nouvelles offres marocaines sont intégrées automatiquement.

Architecture Globale du Système

3.1 Vue d’Ensemble

La plateforme suit une architecture en couches découplées :

TABLE 3.1 – Tableau 4 — Architecture globale du système

Couche	Technologie & Rôle
Frontend	React 18 + TypeScript + Vite — SPA moderne, build optimisé.
Backend API	Django 4.x + Django REST Framework — API RESTful, JWT auth.
Base de données	SQLite (développement) → PostgreSQL (production cible).
Moteur IA/NLP	scikit-learn (TF-IDF), heuristiques Python, Gemini API (optionnel).
Scraping	requests + BeautifulSoup4 — multi-sources marocaines.
Communication	JSON sur HTTPS, CORS configuré, Axios côté client.

3.2 Découpage Backend par Applications Django

Le backend est organisé en quatre applications distinctes : **users/** (gestion des comptes, authentification, rôles, profils étudiants et entreprises, upload CV) ; **offers/** (gestion des offres de stage/emploi, dépôt et suivi des candidatures, déclenchement scraping) ; **ai/** (moteur de recommandations, génération d’insights de carrière, assistants IA) ; **core/** (configuration globale, middlewares, utilitaires transversaux).

3.3 Flux de Données Principal

Flux recommandation (séquence simplifiée) :

1. Étudiant uploadé son CV (PDF/DOCX) via l'API /me/cv.
2. Backend extrait le texte brut du CV (PyPDF2 / python-docx).
3. Le moteur NLP parse les compétences, projets et niveau (heuristique ou Gemini).
4. Le profil enrichi est vectorisé (TF-IDF) et comparé aux offres actives.
5. Les scores de matching sont calculés (sémantique + skills + contexte).
6. Le frontend reçoit le top-N offres avec scores, skills alignées et insights.
7. L'étudiant peut postuler, générer une lettre ou simuler un entretien.

3.4 Justification des Choix Architecturaux

Le découpage en applications Django réduit le couplage inter-modules, facilite les tests unitaires par domaine, et simplifie les évolutions futures. La séparation stricte frontend/backend via API REST permet d'envisager des clients alternatifs (mobile, extension navigateur) sans modifier le backend. Le choix de SQLite en développement accélère le setup local, avec une migration transparente vers PostgreSQL en production via l'ORM Django.

Conception Backend — Django REST Framework

4.1 Modèle de Données Utilisateurs

Le modèle User étend AbstractUser de Django avec un champ role (student / company / admin). Deux profils spécialisés sont liés en OneToOne : **StudentProfile** (niveau d'études, compétences, biographie, préférences de localisation et type de poste, chemin vers le CV uploadé) et **CompanyProfile** (secteur d'activité, description, site web, taille, localisation).

4.2 API d'Authentification et Profil

TABLE 4.1 – Tableau 5 — Endpoints d'authentification et profil

Endpoint	Description
POST /api/auth/register/	Inscription (student ou company)
POST /api/auth/login/	Connexion — retourne access + refresh tokens (JWT)
POST /api/auth/token/refresh/	Renouvellement du token d'accès
GET/PUT /api/users/me/	Profil de l'utilisateur connecté
GET/PUT /api/users/me/student/	Profil étudiant détaillé
GET/PUT /api/users/me/company/	Profil entreprise détaillé
POST /api/users/me/cv/	Upload CV (PDF/DOCX, max 5 Mo)

4.3 Domaine Offres et Candidatures

Les entités principales du domaine recrutement sont : **InternshipOffer** (titre, description, compétences requises en JSON, localisation, type stage/emploi/alternance, statut active/paused/archived, source manuelle ou scrapée, date de publication) et **Application** (référence à l'offre et à l'étudiant, statut pending/accepted/rejected/withdrawn, date de dépôt, contrainte d'unicité).

TABLE 4.2 – Tableau 6 — Endpoints offres et candidatures

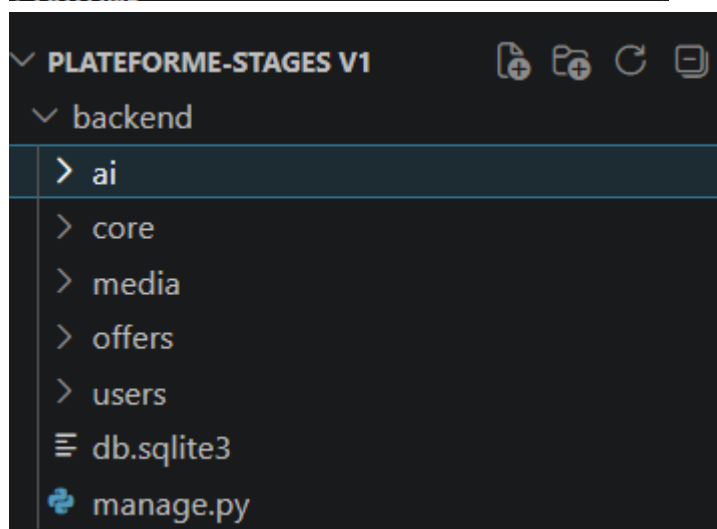
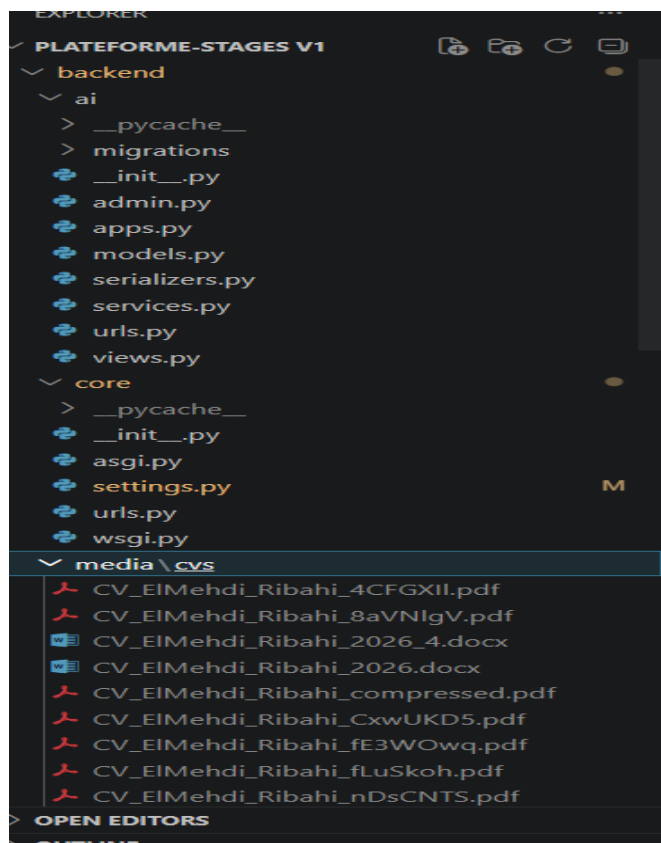
Endpoint	Description
GET /api/offers/	Liste des offres actives (filtrage, pagination)
POST /api/offers/	Créer une offre (Entreprise)
GET/PUT/DELETE /api/offers/{id}/	CRUD offre (propriétaire uniquement)
POST /api/offers/{id}/apply/	Postuler à une offre (Étudiant)
GET /api/applications/received/	Candidatures reçues (Entreprise)
PATCH /api/applications/{id}/status/	Accepter/Refuser une candidature
DELETE /api/applications/{id}/	Retirer sa candidature (si pending)

4.4 Système de Permissions Granulaires

Des classes de permissions personnalisées DRF assurent la sécurité logique : **IsStudent** (réserve les actions candidat aux utilisateurs de rôle 'student') ; **IsCompany** (réserve les actions recruteur aux utilisateurs de rôle 'company') ; **IsOfferOwner** (garantit qu'une entreprise ne peut modifier que ses propres offres) ; **IsApplicationOfferOwner** (garantit qu'une entreprise ne voit que les candidatures pour ses offres).

4.5 Sérialisation et Validation

Les serializers DRF assurent la validation des données entrantes (types, longueurs, formats), la transformation des objets Django en JSON, et la gestion des relations (nested serializers pour les profils). Des validations métier custom sont implémentées : vérification unicité candidature, validation format CV, contrôle taille fichier.



Conception Frontend — React + TypeScript

5.1 Structure du Projet

TABLE 5.1 – Tableau 7 — Structure du projet frontend

Dossier	Rôle
src/pages/	Écrans métier (Dashboard, Profil, Offres, Candidatures, Coaching)
src/components/	Briques UI réutilisables (Cards, Modales, Formulaires, Charts)
src/contexts/	AuthContext — état d'authentification global (React Context API)
src/services/api.ts	Client Axios centralisé avec intercepteurs
src/hooks/	Custom hooks (useRecommendations, useApplications, useProfile)
src/types/	Interfaces TypeScript pour tous les modèles de données

5.2 Gestion de l'Authentification Côté Client

Stockage sécurisé des tokens : access_token et refresh_token en mémoire / sessionStorage. Injection automatique du header Authorization : Bearer via intercepteur Axios. Refresh transparent : intercepteur de réponse sur erreur 401 → appel automatique au refresh endpoint → retry de la requête originale. Déconnexion automatique si le refresh token est expiré.

5.3 Parcours Étudiant

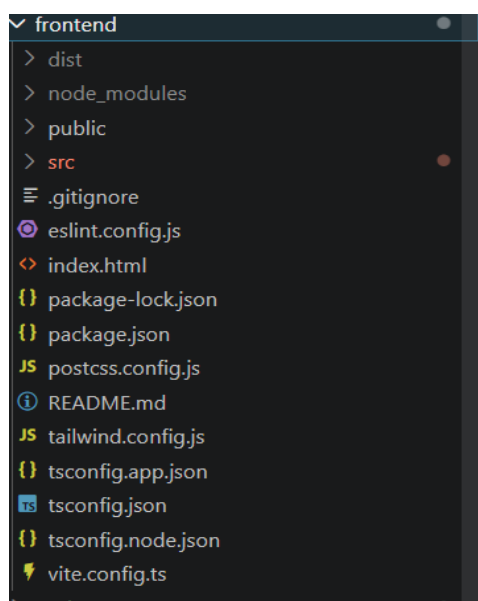
Inscription → Complétion du profil (compétences, préférences, biographie). Upload CV PDF/DOCX → Parsing et affichage du profil extrait. Dashboard Recommandations → Visualisation des top offres avec scores, compétences alignées/manquantes. Vue Insights de Carrière → Forces, compétences prioritaires, villes cibles, plan d'action. Candidature → Suivi des statuts en temps réel. Assistants IA → Génération lettre de motivation + simulateur d'entretien.

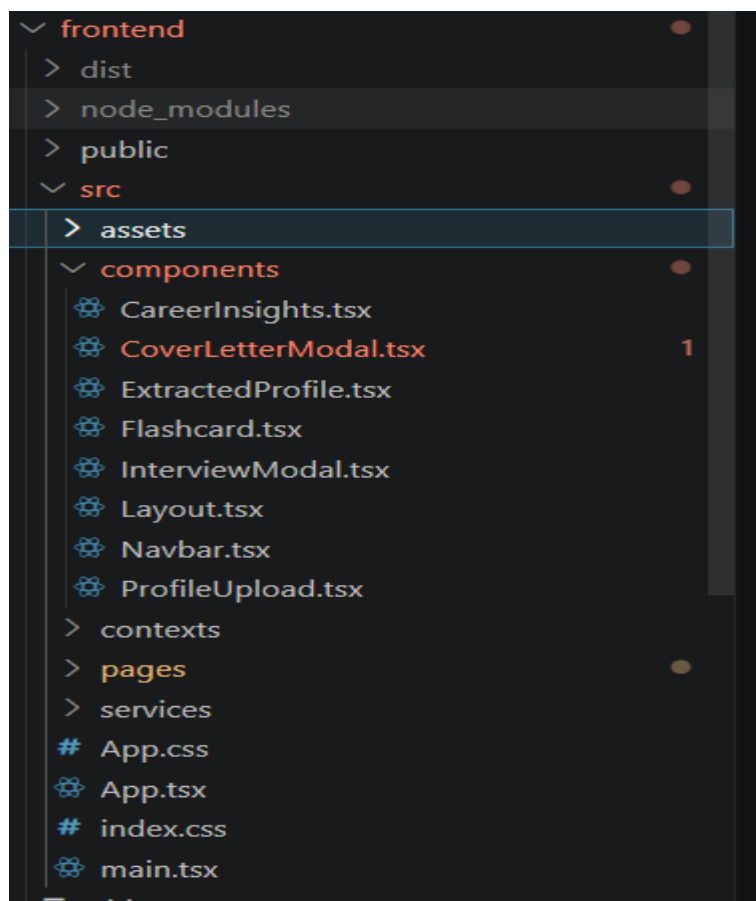
5.4 Parcours Entreprise

Création d'offres avec formulaire enrichi (compétences, localisation, type, description). Tableau de bord des offres publiées avec pilotage des statuts (active/paused/archived). Consultation des candidatures reçues avec filtre par offre et statut. Actions d'acceptation ou de refus avec feedback automatique au candidat.

5.5 Expérience Utilisateur et Design

L'interface met en avant la lisibilité des scores de matching (barres de progression colorées, badges qualité : Excellent / Fort / Prometteur / À explorer), l'explicabilité des matches (liste des compétences alignées et manquantes), et des actions directes (postuler, générer lettre, filtrer les offres). Le design responsive assure une utilisation optimale sur desktop et mobile.





Moteur IA/NLP et Logique de Recommandation

6.1 Pipeline Global de Recommandation

Le moteur de recommandation suit un pipeline en 8 étapes : extraction texte brut du CV (PyPDF2 pour PDF, python-docx pour DOCX) ; normalisation linguistique (minuscules, suppression ponctuation, lemmatisation basique) ; extraction structurée (compétences, niveau d'études, projets, expériences via heuristiques Python ou Gemini API) ; construction du texte enrichi de l'offre (titre + description + compétences requises + contexte métier) ; vectorisation TF-IDF des profils et des offres avec corpus commun ; calcul de la similarité cosinus entre le vecteur profil et chaque vecteur offre ; application des bonus contextuels (overlap skills, préférences localisation/type poste, niveau d'études) ; fusion en score final pondéré → ranking → génération des insights → retour API.

6.2 Formule de Scoring

$$\text{Score Final} = w_1 \times \text{Score_Sémantique} + w_2 \times \text{Score_Skills} + w_3 \times \text{Bonus_Contexte}$$

Score_Sémantique : similarité cosinus TF-IDF profil ↔ offre (poids principal, 60%).
Score_Skills : taux de chevauchement hard skills (compétences en commun / total requis).
Bonus_Contexte : préférences utilisateur (localisation, type poste), niveau études (plafonné). Cette stratégie équilibre robustesse sémantique, pertinence métier et personnalisation utilisateur.

6.3 Explainable AI (XAI)

Chaque recommandation expose les éléments suivants pour assurer la transparence : score global sur 100 avec bande qualité (Excellent ≥ 85 , Fort ≥ 70 , Prometteur ≥ 55 , À explorer < 55) ; liste des compétences alignées entre le profil et l'offre ; liste des compétences manquantes ; résumé explicatif en langage naturel de la raison du score ; indice de confiance basé sur la complétude du profil (CV parsé vs profil manuel).

6.4 Insights de Carrière

Au-delà du matching individuel, le module d'insights agrège les données des top-N recommandations pour fournir une vision macro : forces dominantes du profil (compétences les plus valorisées sur le marché) ; compétences prioritaires à acquérir (gaps récurrents dans les offres bien matchées) ; villes cibles (géolocalisation des meilleures opportunités disponibles) ; track recommandé (orientation métier suggérée ex. Full-Stack, Data, DevOps) ; plan d'action personnalisé (ressources pour combler les compétences manquantes).

6.5 IA Générative Complémentaire

TABLE 6.1 – Tableau 8 — Modules IA génératifs

Module IA	Description
Lettre de motivation	Génération contextualisée par Gemini API : profil étudiant + description offre + ton souhaité → lettre personnalisée en markdown.
Simulateur d'entretien	Gemini joue le rôle du recruteur : génère des questions adaptées à l'offre, évalue les réponses, propose des axes d'amélioration.
Analyse tendances marché	Agrégation des offres scrapées : top compétences, top villes, top secteurs — visualisation sur le dashboard.

Scraping et Intelligence Marché

7.1 Architecture du Module de Scraping

Le module de scraping est conçu pour alimenter automatiquement la base d'offres depuis des sources marocaines et internationales pertinentes. Il est déclenché à la demande par l'administrateur ou planifiable via cron. Sources marocaines prioritaires : Rekrute.com, Emploi.ma, MarocAnnonces, LinkedIn Maroc (via parsing public). Sources internationales de fallback : offres francophones pertinentes pour les profils marocains. Technologie : requests + BeautifulSoup4 pour le parsing HTML, avec gestion des user-agents et des délais pour respecter les politiques robots.txt.

7.2 Pipeline de Normalisation

Extraction des champs bruts (titre, description, entreprise, localisation, type, date). Normalisation (nettoyage HTML, standardisation des formats de date, correction encodages). Détection de pertinence Maroc (filtre sur localisation et mots-clés contextuels). Extraction automatique des compétences via le moteur NLP (même pipeline que pour les CV). Upsert en base (insertion ou mise à jour selon une clé de déduplication titre + entreprise + date).

7.3 Résilience et Continuité

Stratégie de résilience multi-niveaux : en cas d'échec réseau → retry avec backoff exponentiel (3 tentatives max) ; en cas d'échec global du scraping → activation du fallback local (dataset d'offres cohérentes pré-chargé) ; continuité fonctionnelle garantie (la plateforme reste opérationnelle même sans nouvelles offres scrapées) ; logging détaillé (chaque

session de scraping produit un rapport : sources contactées, offres extraites, erreurs).

7.4 Analytics Marché

Les données scrapées alimentent un module d'analytics accessible via le dashboard admin et la vue tendances marché pour les étudiants : top 10 compétences les plus demandées sur le marché marocain (période glissante 30 jours) ; top villes par volume d'offres actives ; top plateformes de recrutement par taux de nouvelles offres ; évolution temporelle des secteurs les plus actifs.

Sécurité, Performance et Scalabilité

8.1 Sécurité — Implémentations Actuelles

Authentification JWT (JSON Web Tokens) : access token courte durée + refresh token longue durée, stockés côté client. Permissions DRF granulaires : contrôle d'accès par rôle et par propriété des ressources. CORS configuré : liste blanche des origines autorisées en production. Validation côté serializers : typage strict, longueurs max, formats attendus. Protection CSRF : désactivée sur les endpoints API stateless (JWT), mais active pour les vues Django classiques.

8.2 Points à Renforcer pour la Production

TABLE 8.1 – Tableau 9 — Points de sécurité à renforcer

Point	Action Recommandée
Security Headers	Ajout de Strict-Transport-Security, X-Frame-Options, Content-Security-Policy via django-security ou middleware custom.
Secrets Management	Migration vers variables d'environnement (django-environ) ou vault (HashiCorp Vault / AWS Secrets Manager).
ALLOWED_HOSTS	Politique stricte en production, sans wildcard.
Audit OWASP API Top 10	Vérification injection, broken object auth, excessive data exposure, rate limiting.
Traçabilité IA	Logging des prompts et réponses Gemini pour audit et conformité (RGPD Maroc - Loi 09-08).

8.3 Performance

Le risque principal de performance identifié est le traitement NLP synchrone lors de l'upload de CV ou du refresh des recommandations. Les solutions envisagées pour la production : jobs asynchrones via Celery avec Redis/RabbitMQ comme broker de messages ; files d'attente prioritaires (recommandations en file haute priorité, scraping en file basse) ; cache partiel (mise en cache Redis des vecteurs TF-IDF pour les offres non modifiées, TTL 1 heure) ; retry / backoff automatique pour les appels à l'API Gemini en cas de timeout.

8.4 Scalabilité

Base de données : migration SQLite → PostgreSQL 15+ avec indexation optimisée sur les champs de recherche. Stockage médias : migration vers stockage objet (AWS S3 / Cloudflare R2) pour les CV uploadés. Observabilité : logs structurés (JSON) + métriques applicatives (Prometheus + Grafana). CI/CD : pipeline GitHub Actions pour tests automatisés, linting, build et déploiement continu. Containerisation : Dockerisation du backend et du frontend pour déploiement reproductible.

Validation, Limites et Discussion

9.1 Validation Qualitative

La plateforme a été validée qualitativement sur l'ensemble des scénarios métier clés :

TABLE 9.1 – Tableau 10 — Résultats de validation par scénario

Scénario	Résultat
Matching étudiant	Les profils bien renseignés (CV + compétences) reçoivent des recommandations cohérentes avec leur niveau et leurs compétences.
Parcours entreprise	Création d'offres, consultation et mise à jour des statuts de candidatures fonctionnels.
Outils IA	Génération de lettres de motivation contextualisées. Simulateur d'entretien opérationnel.
Scraping	Offres marocaines récupérées et normalisées. Fallback activé en cas d'échec réseau.
Sécurité	Tests manuels de contrôle d'accès : accès refusé aux ressources d'autres utilisateurs.

9.2 Limites Actuelles

Couverture de tests insuffisante : absence de tests unitaires systématiques et de tests d'intégration automatisés. Matching principalement lexical/sémantique léger : dépendance à la qualité textuelle des CV et des offres. Absence de signaux comportementaux : pas de collaborative filtering basé sur l'historique des clics ou candidatures. Scalabilité non

validée : tests de charge non effectués (charge cible : 1 000 utilisateurs simultanés). Données limitées : scraping initial sur un volume restreint d’offres marocaines.

9.3 Discussion Scientifique

Le choix de TF-IDF + similarité cosinus comme socle du moteur de recommandation est justifié par plusieurs facteurs : coût computationnel minimal (pas de GPU requis), interprétabilité directe des résultats (poids TF-IDF visualisables), rapidité de déploiement, et robustesse suffisante pour un dataset de taille modérée (< 10 000 offres).

Les limites de cette approche sont connues : incapacité à gérer les synonymes (ex. « développeur » $\neq$ « ingénieur logiciel » pour TF-IDF), sensibilité à la longueur des documents, et absence de compréhension contextuelle profonde. L’évolution naturelle vers Sentence-BERT permettrait de dépasser ces limitations, avec un trade-off computationnel acceptable en production.

Métriques d’évaluation formelle recommandées pour la V2 : Precision@K (parmi les K premières recommandations, quelle proportion est réellement pertinente ?) ; Recall@K (quelle proportion des offres pertinentes est capturée dans les K premières ?) ; nDCG — Normalized Discounted Cumulative Gain (mesure l’ordre de pertinence du ranking) ; MRR — Mean Reciprocal Rank (position moyenne de la première recommandation pertinente).

Synthèse comparative et Positionnement

10.1 Tableau récapitulatif

TABLE 10.1 – Tableau 11 — Synthèse comparative des six sources de l'état de l'art

Source	Concept	Dataset	Algorithme	Performance	Quartile
Isinkaye et al. (2015) [1]	Content-based / Collaborative / Hybride	Benchmark multi-sources	Multiple	Référence	Q1
Siting et al. (2012) [2]	Job Recommender Systems Survey	Synthèse	Taxonomie	Référence	Q2
Al-Otaibi & Ykhlef (2012) [3]	Survey systèmes RH	Synthèse	Taxonomie	Référence	Q2
Yi et al. (2020) [4]	TF-IDF + Cosine Similarity	Offres/CV réels	TF-IDF + Cosine	$F1 \approx 87 \%$	Q2
Kops et al. (2013) [5]	Resume Parsing automatisé	CV réels	NER + Règles	Précision $> 80 \%$	Q2
Reimers & Gurevych (2019) [6]	Sentence-BERT embeddings	NLI + STS	SBERT	State-of-the-art	Q1

10.2 Lacunes identifiées et axes d'innovation

Lacune 1 — Absence de plateforme orientée marché marocain. Aucun des travaux examinés ne cible explicitement le marché de l'emploi étudiant marocain (Rekrute, Emploi.ma), avec ses spécificités linguistiques (français/arabe/darija) et sectorielles. La plateforme innove en intégrant un scraping dédié et une logique métier adaptée.

Lacune 2 — Absence d'approche intégrée parsing + matching + coaching IA. Les solutions existantes traitent séparément le resume parsing, le matching et les outils d'aide à la candidature. La plateforme propose une architecture unifiée en trois couches (backend API, moteur NLP, interface coaching) couvrant les trois dimensions

simultanément.

Lacune 3 — Absence d’Explainable AI (XAI) dans les plateformes académiques.

Aucune solution académique recensée n’intègre un niveau d’explicabilité aussi granulaire (scores par dimension, compétences alignées/manquantes, résumé en langage naturel). Cette dimension de transparence constitue une contribution originale du projet.

10.3 Perspectives de Recherche Futures

Apprentissage fédéré pour la recommandation distribuée : les modèles actuels sont entraînés de manière centralisée. Une approche d’apprentissage fédéré permettrait d’agréger les connaissances de milliers de profils sans transmettre de données personnelles. Détection de biais algorithmiques : aucun des articles retenus ne teste la présence de biais de genre ou de région dans les recommandations — une dimension critique pour un système d’aide à l’emploi équitable. Migration vers Sentence-BERT pour un matching sémantique profond. Extension vers un client mobile natif (React Native) et une extension navigateur.

Conclusion et Perspectives

11.1 Conclusion

Ce Projet de Fin d'Année a permis de concevoir et d'implémenter une plateforme intelligente de recommandation de stages et d'emplois, fonctionnelle et cohérente avec les objectifs d'un PFA de cycle ingénieur. Les résultats obtenus démontrent la faisabilité d'un système de recommandation explicable et utile avec des technologies open-source accessibles.

Les principaux apports du projet sont : une architecture backend API propre et sécurisée (Django REST Framework + JWT) ; un frontend réactif et modulaire (React + TypeScript) ; un moteur NLP hybride interprétable (TF-IDF + règles métier) ; des outils d'IA générative augmentant la valeur utilisateur (lettre de motivation, simulateur d'entretien) ; un module de scraping résilient orienté marché marocain.

Valeur créée par le projet : pour les étudiants (réduction du temps de recherche, recommandations personnalisées, coaching IA) ; pour les entreprises (meilleure visibilité des candidats qualifiés, pilotage centralisé) ; pour l'écosystème (veille automatique du marché marocain de l'emploi étudiant).

11.2 Perspectives d'Évolution

TABLE 11.1 – Tableau 12 — Roadmap d'évolution

Horizon	Évolutions Planifiées
Court terme (3–6 mois)	Tests automatisés (pytest-django, Jest), passage PostgreSQL, dockerisation, CI/CD GitHub Actions.
Moyen terme (6–12 mois)	Architecture asynchrone (Celery), moteur hybride (content-based + collaborative filtering), évaluation formelle Precision@K / nDCG.
Long terme (12+ mois)	Migration vers Sentence-BERT pour le matching sémantique profond, tableau de bord entreprise avancé (analytics candidatures), conformité RGPD / Loi 09-08, modèle SaaS avec plans tarifaires.

Références bibliographiques

1. F. O. Isinkaye, Y. O. Folajimi, and B. A. Ojokoh, “Recommendation systems : Principles, methods and evaluation,” *Egyptian Informatics Journal*, vol. 16, no. 3, pp. 261–273, 2015.
2. K. Siting, W. Wenxing, Z. Ning, and Y. Fan, “Job recommender systems : A survey,” *Proc. 7th Int. Conf. Computer Science & Education (ICCSE)*, 2012, pp. 920–924.
3. T. Al-Otaibi and M. Ykhlef, “A survey of job recommender systems,” *International Journal of Physical Sciences*, vol. 7, no. 29, pp. 5127–5142, 2012.
4. D. Yi, C. Wei, and J. Ruifeng, “A content-based job recommendation system using TF-IDF and cosine similarity,” *Proc. Int. Conf. Machine Learning and Computing*, 2020.
5. E. Kops, G. Kessler, and T. Dürschner, “Automated resume information extraction to support the matching of job seekers and vacancies,” *Proc. 21st European Conf. Information Systems (ECIS)*, 2013.
6. N. Reimers and I. Gurevych, “Sentence-BERT : Sentence embeddings using Siamese BERT-networks,” *Proc. EMNLP-IJCNLP 2019*, pp. 3982–3992.
7. J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT : Pre-training of deep bidirectional transformers for language understanding,” *Proc. NAACL-HLT 2019*, pp. 4171–4186.
8. T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, “Distributed representations of words and phrases and their compositionality,” *Proc. NIPS*, 2013, pp. 3111–3119.
9. Django Software Foundation, “Django REST Framework Documentation,” Version 3.14, 2024. Disponible : <https://www.django-rest-framework.org/>
10. Facebook Inc., “React Documentation,” Version 18, 2024. Disponible : <https://reactjs.org/>

11. F. Pedregosa et al., “Scikit-learn : Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
12. Google LLC, “Gemini API Documentation,” 2024. Disponible : <https://ai.google.dev/docs>

— *Fin du Rapport* —

EMSI Casablanca | PFA 2025–2026 | Ayman Ould-Naghouch & El Mehdi Ribahi